

Hyper Tree Network

Dr.P.Ganesh Kumar
Anna University

Introduction

The Binary Tree Network provides a simple hierarchical structure but suffers from **large communication delays** between distant leaf nodes and **heavy traffic at the root**. The Hypercube Network offers **short communication paths** but requires a large number of communication links as the system size increases.

The **Hypertree Network** combines the strengths of both topologies. It retains the **hierarchical structure of a binary tree** while adding **horizontal links** between processors at the same level. These additional links significantly reduce communication distance and improve load balancing.

Consequently, the Hypertree Network provides:

- Lower communication latency
- Higher throughput
- Better scalability
- Improved fault tolerance
- Efficient routing

It is widely studied in **parallel processing, distributed computing, Network-on-Chip (NoC), cloud computing, and high-performance computing (HPC)**.

Definition

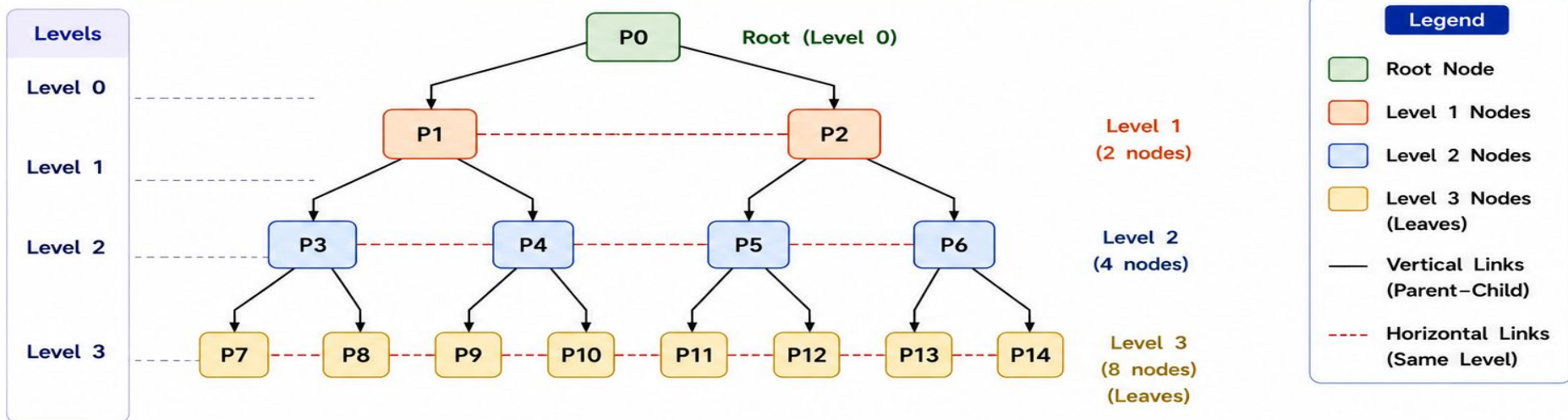
A **Hypertree Network** is a hierarchical interconnection network that extends a binary tree by introducing **additional horizontal connections between processors at the same level**, thereby reducing communication distance and improving overall network performance.

Characteristics

- Hierarchical organization
- Tree-based structure
- Additional horizontal links
- Multiple communication paths
- Lower network diameter
- Improved fault tolerance
- Better scalability

Figure 4.11 – Basic Structure of Hypertree Network

A Hypertree Network is a hierarchical extension of a binary tree in which additional **horizontal links** connect nodes at the same level.



Key Features

- ✓ Hierarchical organization like a binary tree.
- ✓ Each node (except the root) has one parent and up to two children.
- ✓ Additional horizontal links connect neighboring nodes at the same level.
- ✓ Multiple paths exist between any two nodes, reducing communication distance.
- ✓ Improves performance, scalability and fault tolerance.

Example (Height = 3)

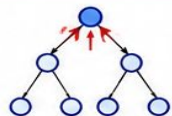
- | | |
|-------------------------|--------------------------------|
| • Number of Levels | : 4 (0, 1, 2, 3) |
| • Total Number of Nodes | : $2^{h+1} - 1 = 2^4 - 1 = 15$ |
| • Leaf Nodes (Level 3) | : $2^h = 2^3 = 8$ |
| • Internal Nodes | : $2^h - 1 = 2^3 - 1 = 7$ |
| • Horizontal Links | : $(2 + 4 + 8) = 14$ |
| • Vertical Links | : $(N - 1) = 14$ |

Provides multiple communication paths

Hypertree networks combine the hierarchical structure of trees with the short-path advantage of horizontal connections, offering efficient and scalable communication.

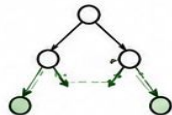
2. Need for Hyper Tree Network

Although the binary tree network is simple and easy to implement, it suffers from several limitations that affect performance, scalability and reliability in large parallel systems.



1 Root Bottleneck

- The root node handles most of the communication.
- This causes congestion and limits throughput.



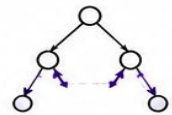
2 Long Communication Paths

- Communication between two leaf nodes must pass through the root.
- This results in large latency and high delay.



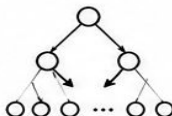
3 Low Fault Tolerance

- Failure of an internal node disconnects an entire subtree.
- The system becomes vulnerable.



4 Limited Parallel Communication

- Only one path exists between many pairs of nodes.
- This limits parallel data transfer capability.



5 Poor Scalability

- As the number of processors increases, communication delay grows.
- The binary tree becomes inefficient for large systems.



These limitations motivate the need for an enhanced structure that reduces communication distance, balances load and improves reliability. Hyper Tree Network addresses these issues effectively.

4.2 Why Hypertree Network?

Hypertree network enhances the binary tree by adding horizontal links between nodes at the same level. This leads to significant improvements.

Problem in Binary Tree

Root Bottleneck

Heavy traffic at root causes congestion.



Long Communication Paths

Paths between distant nodes are long.



Low Fault Tolerance

Failure of a node breaks an entire subtree.



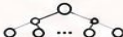
Limited Parallel Communication

Only one path for many node pairs.



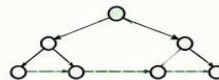
Poor Scalability

Delay increases as system size grows.



Hypertree Solution

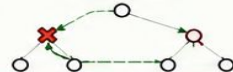
Horizontal links distribute traffic among nodes at the same level.



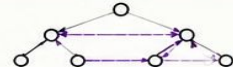
Horizontal links provide shorter alternative routes.



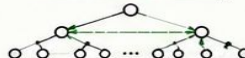
Multiple alternative paths exist due to horizontal links.



Multiple paths enable simultaneous communication between nodes.



Shorter paths and distributed traffic ensure better scalability.



Result / Improvement

Load is balanced and root congestion is significantly reduced.



Communication distance and latency are reduced.



System becomes more reliable and fault tolerant.



Supports more parallel communications.



Better scalability for large parallel systems.



Overall Impact



Shorter paths



Balanced load



Higher throughput



Better reliability



Improved scalability

Hypertree network combines the hierarchical structure of tree with the short-path advantage of horizontal connections, offering efficient, scalable and reliable communication.

3. Organization of Hypertree Network

A hypertree network is a hierarchical interconnection network that extends a binary tree by adding **horizontal links** between nodes at the same level.

Key Points

- Structure is organized in levels like a binary tree.
- Each node (except root) has one parent and up to two children.
- Additional horizontal links connect neighboring nodes at the same level.
- Leaves are at the lowest level.

Properties

- Total levels : $h + 1$ (0 to h)
- Total nodes : $N = 2^{h+1} - 1$
- Nodes at level i : 2^i ($0 \leq i \leq h$)
- Leaves (at level h) : 2^h
- Internal nodes : $N - 2^h$
- Horizontal links : $2^{h+1} - 2$
- Vertical (tree) links : $N - 1$

Example (Height $h = 3$)

- Levels : 0, 1, 2, 3 $\rightarrow$ 4 levels
- Total nodes : $2^{3+1} - 1 = 15$
- Leaves (level 3) : $2^3 = 8$ (P7 to P14)
- Internal nodes : $15 - 8 = 7$ (P0 to P6)
- Horizontal links : $(2 + 4 + 8) - 0 = 14$
- Vertical links : $15 - 1 = 14$

Characteristics

- ✓ Hierarchical and regular structure.
- ✓ Shorter communication paths than binary tree.
- ✓ Multiple routing options improve reliability.
- ✓ Well-suited for scalable parallel systems.
- ✓ Combines advantages of trees and meshes.

Legend

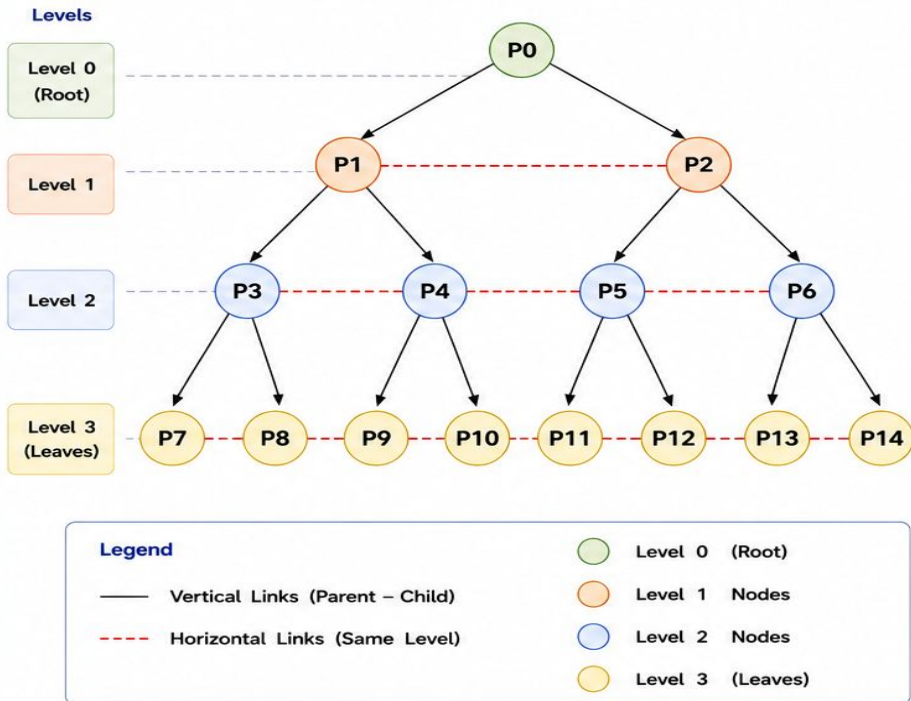


Hypertree organization combines the hierarchical order of trees with the connectivity of meshes at each level, resulting in efficient, scalable and reliable communication.

Figure 4.3 – Organization of Hypertree

Professional illustration showing:

- Vertical tree connections
- Horizontal links at each level
- Processor labels



Hypertree organizes processors in hierarchical levels with both vertical tree connections and horizontal links at each level, providing multiple paths for communication and improved performance.

4. Components of Hypertree Network

A hypertree network consists of processing elements connected in a hierarchical (tree) structure with additional horizontal links at each level. Each node includes several key components that enable computation, storage, and communication.

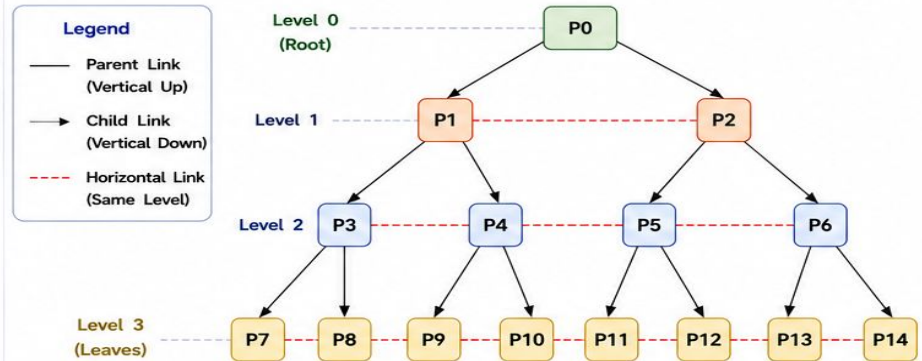
Component	Description
1 Processing Element (PE) 	- Core unit that performs computations. - Executes instructions and processes data. - Communicates with other nodes.
2 Parent Link 	- Connects a node to its parent. - Used for upward communication in the tree. - Carries data, control, and acknowledgment messages.
3 Child Links 	- Connect a node to its children. - Used for downward communication. - Multiple child links allow parallel data transfer.
4 Horizontal Links (Same-Level Links) 	- Connect neighboring nodes at the same level. - Reduce communication distance. - Provide multiple routing options and load balancing.
5 Communication Interface 	- Handles sending and receiving of messages. - Implements routing, flow control and error detection. - Converts data to/from the network format.
6 Local Memory 	- Stores local data and instructions. - Used for caching and intermediate results. - Reduces access latency to shared data.
7 Control Logic 	- Manages internal operations of the node. - Handles synchronization, routing decisions, and fault detection. - Coordinates communication with neighbors.



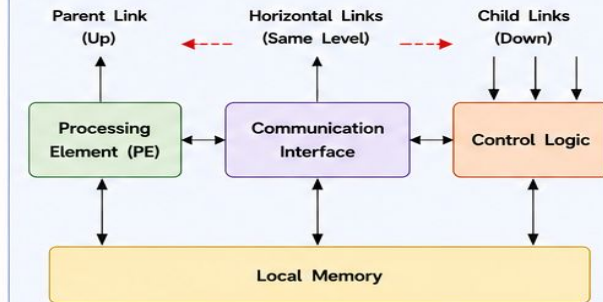
These components work together to provide efficient computation, memory storage, and multi-path communication in a hypertree network.

4.4 Components of Hypertree Network?

Each node in a hypertree network contains the following components and is connected to other nodes through vertical and horizontal links.



Internal Structure of a Node



Component Summary

- 1 Processing Element (PE)
- 2 Parent Link (Up)
- 3 Child Links (Down)
- 4 Horizontal Links (Same Level)
- 5 Communication Interface
- 6 Local Memory
- 7 Control Logic



These components enable each node to compute, store data, and communicate efficiently with its parent, children, and neighbors at the same level.

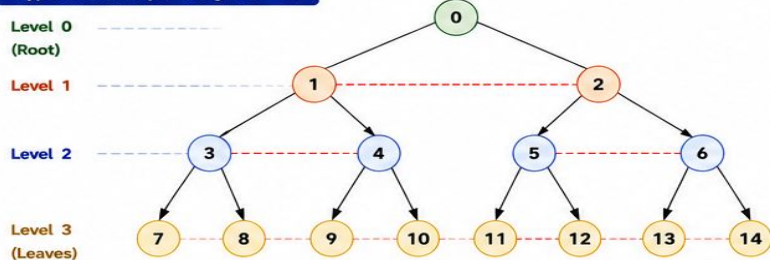
4. Node Addressing Scheme

In a hypertree network, each node (processor) is assigned a unique address (ID) using binary tree indexing. The addressing follows the same rules as a complete binary tree. Horizontal links are between nodes at the same level and do not change the addressing.

Addressing Rules (Binary Tree Indexing)

Relationship	Formula (for node i)	Description
Root	$i = 0$	The root node has address 0.
Parent	$\lfloor (i - 1) / 2 \rfloor$	Integer division (floor) gives the parent.
Left Child	$2i + 1$	Address of the left child.
Right Child	$2i + 2$	Address of the right child.
Level of Node i	$\lfloor \log_2(i + 1) \rfloor$	Level (root is level 0).
Total Nodes (height h)	$N = 2^{h+1} - 1$	Total number of nodes in a hypertree of height h .

Hypertree Example (Height $h = 3$)



Observations

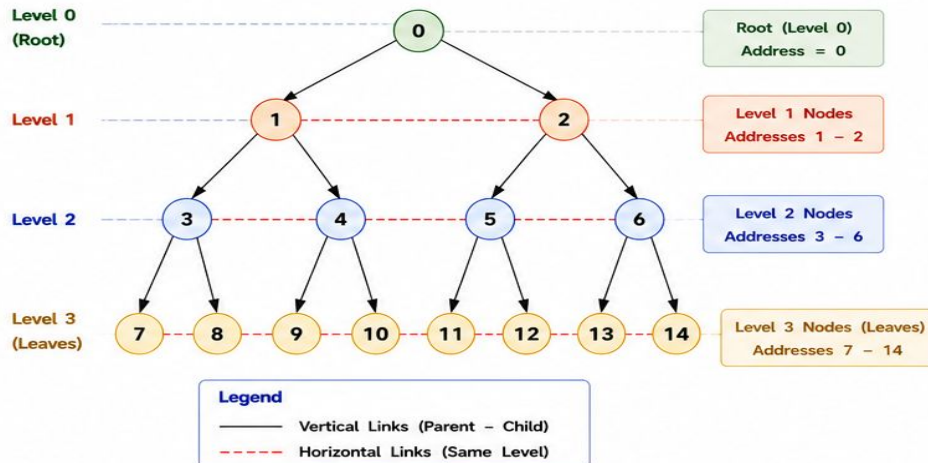
- Addresses increase from left to right at each level.
- Nodes at the same level are connected by horizontal links.
- The addressing scheme enables easy computation of parent and children.

Example Calculations

- Parent of node 10: $\lfloor (10 - 1) / 2 \rfloor = \lfloor 9 / 2 \rfloor = 4$
- Left child of node 4: $2(4) + 1 = 9$
- Right child of node 4: $2(4) + 2 = 10$
- Level of node 13: $\lfloor \log_2(13 + 1) \rfloor = \lfloor \log_2 14 \rfloor = 3$
- Parent of root (0): Not defined

Figure 4.5 Addressing Scheme

A visual representation of the node addressing scheme in a hypertree network (height $h = 3$).



Addressing Summary Table

Node i	Parent $\lfloor (i-1)/2 \rfloor$	Left Child $2i+1$	Right Child $2i+2$	Level $\lfloor \log_2(i+1) \rfloor$
0	-	1	2	0
1	0	3	4	1
2	0	5	6	1
3	1	7	8	2
4	1	9	10	2
5	2	11	12	2
6	2	13	14	2
...	...	...	...	...
14	6	-	-	3

Key Points

- Each node has a unique address using binary tree indexing.
- Parent and child addresses can be computed using simple formulas.
- Addresses increase from left to right at each level.
- Horizontal links connect consecutive addresses at the same level.
- This scheme enables efficient routing and easy neighbor identification.



This addressing scheme provides a systematic way to identify parents, children, levels, and neighbors, which simplifies routing and communication in the hypertree.



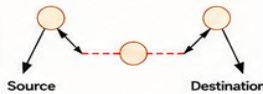
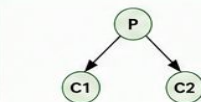
The addressing scheme follows binary tree indexing, enabling efficient computation of relationships and simplifying communication in the hypertree network.

6. Communication in Hypertree Network

In a hypertree network, data can be exchanged between any two nodes using vertical (parent-child) links, horizontal (same-level) links, or a combination of both. The presence of horizontal links reduces communication distance and avoids unnecessary traversal to the root.

Types of Communication

- | 1 | Vertical Communication (Parent - Child) |
|---|---|
| | <ul style="list-style-type: none">Occurs between a node and its parent or child.Used for moving up or down in the tree. |
| 2 | Horizontal Communication (Same Level) |
| | <ul style="list-style-type: none">Occurs between neighboring nodes at the same level.Uses horizontal links. |
| 3 | Hybrid Communication (Vertical + Horizontal) |
| | <ul style="list-style-type: none">Combination of vertical and horizontal movements.Used to find shorter and efficient communication paths. |



Key Points

- ✓ Direct parent-child links are used for vertical movement.
- ✓ Horizontal links connect adjacent nodes at the same level.
- ✓ Communication between distant nodes uses a combination of vertical and horizontal links.
- ✓ Multiple paths exist between nodes, improving reliability and load balancing.
- ✓ Horizontal links significantly reduce communication distance and delay.

Benefits

- ✓ Shorter communication paths
- ✓ Reduced latency
- ✓ Better bandwidth utilization
- ✓ Multiple alternative paths
- ✓ Improved fault tolerance
- ✓ Scalability and higher throughput

Example: Communication Between Two Leaves

Without Horizontal Links (Binary Tree)

Path from Node 9 to Node 14
 $9 \rightarrow 4 \rightarrow 1 \rightarrow 0 \rightarrow 2 \rightarrow 6 \rightarrow 14$
Total Hops = 6

With Horizontal Links (Hypertree)

Path from Node 9 to Node 14
 $9 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 14$
Total Hops = 4

Improvement

- Paths are shorter.
- Less congestion at the root.
- Lower communication delay.
- Higher overall performance.



Hypertree networks enable efficient communication by using vertical, horizontal, or hybrid paths, resulting in shorter routes, lower latency, and better system performance.

4.6 Communication Paths

Comparison of communication between two nodes in a binary tree and in a hypertree. Hypertree uses horizontal links to reduce the number of hops.

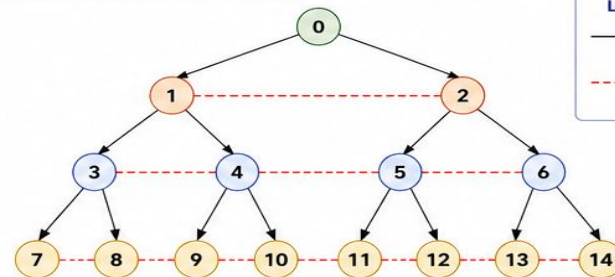
Hypertree Example (Height $h = 3$)

Level 0 (Root)

Level 1

Level 2

Level 3 (Leaves)

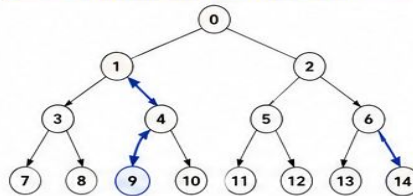


Legend

- Vertical Link (Parent - Child)
- - - Horizontal Link (Same Level)

Example Communication: From Node 9 to Node 14

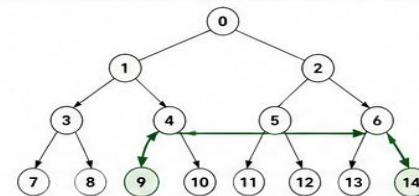
A. Binary Tree (Without Horizontal Links)



Path: $9 \rightarrow 4 \rightarrow 1 \rightarrow 0 \rightarrow 2 \rightarrow 6 \rightarrow 14$

Total Hops: 6

B. Hypertree (With Horizontal Links)



Path: $9 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 14$

Total Hops: 4

Observations

- In a binary tree, communication must go up to the root and then down to the destination.
- In a hypertree, horizontal links allow movement at the same level, avoiding the root.
- Hypertree communication paths are shorter, faster, and more efficient.

Key Advantages

- ✓ Reduced number of hops
- ✓ Lower communication delay
- ✓ Better bandwidth utilization
- ✓ Improved system performance



Hypertree networks provide multiple communication paths using vertical and horizontal links, resulting in shorter distances, reduced congestion, and improved reliability.

7. Routing Algorithm

Routing in a hypertree network is based on the principle of moving the message up the tree toward the Least Common Ancestor (LCA) of the source and destination, and then down the tree to the destination. Horizontal links are used at each level to minimize the number of hops.

Routing Principles

- ✓ **Move Up:** From the source node up to the LCA using parent links.
- ✓ **Move Across:** At the LCA level, use horizontal links to reach the destination subtree.
- ✓ **Move Down:** From the LCA down to the destination node using child links.
- ✓ This strategy guarantees the shortest path in terms of hop count.

Routing Algorithm (Source S to Destination D)

- 1 If $S = D$, stop.
- 2 Compute the level of S and D.
- 3 While $\text{level}(S) > \text{level}(D)$:
 - Move S up to its parent.
- 4 While $\text{level}(D) > \text{level}(S)$:
 - Move D up to its parent.
- 5 While S and D are not on the same level:
 - Move both S and D up to their parents.
- 6 While $S \neq D$:
 - If S and D are at the same level, move horizontally toward each other (left or right whichever is nearer).
 - Otherwise, move both up to parents.
- 7 Now S is at LCA. From LCA, move down to D using child links.

Example (Height $h = 3$)

Source (S) : Node 9
 Destination (D) : Node 14
 Least Common Ancestor (LCA) : Node 0 (Root)
 Routing Path : $9 \rightarrow 4 \rightarrow 1 \rightarrow 0 \rightarrow 2 \rightarrow 6 \rightarrow 14$
 Total Hops : 6

Pseudocode

```

ROUTE(S, D)
1  if S = D return
2  while level(S) > level(D)
3    S ← parent(S)
4  while level(D) > level(S)
5    D ← parent(D)
6  while parent(S) ≠ parent(D)
7    S ← parent(S)
8    D ← parent(D)
9  while S ≠ D
10   if S and D are at same level
11     move horizontally towards each other
12   else
13     S ← parent(S)
14     D ← parent(D)
15  From LCA(S, D), move down to D
    
```

Characteristics

- ✓ Guaranteed shortest path in terms of hops.
- ✓ Uses both vertical and horizontal links.
- ✓ Scalable and efficient for large networks.
- ✓ Simple to implement and distributed.

Figure 4.7 Routing in Hypertree

Example of routing from Source Node 9 to Destination Node 14 in a hypertree of height $h = 3$.

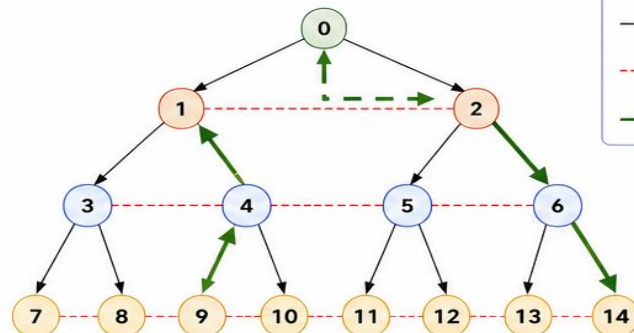
Hypertree Structure (Height $h = 3$)

Level 0
(Root)

Level 1

Level 2

Level 3
(Leaves)



Legend

- Vertical Link (Parent – Child)
- - - Horizontal Link (Same Level)
- Routing Path

Routing Steps: From Node 9 to Node 14

Step	Action	From Node	To Node	Link Type	Description
1	Move Up	9	→ 4	Vertical Up	Move up to parent
2	Move Up	4	→ 1	Vertical Up	Move up to parent
3	Move Up	1	→ 0	Vertical Up	Move up to root
4	Move Across	0	→ 2	Horizontal	Move to destination subtree
5	Move Down	2	→ 6	Vertical Down	Move down to child
6	Move Down	6	→ 14	Vertical Down	Move down to destination

Observation

- The path goes up to the root (or LCA) and then down.
- Only one horizontal move is needed at the LCA level.
- Total hops = 6 (optimal path).
- Horizontal links significantly reduce the number of hops.

Key Advantages

- ✓ Shortest path in terms of hops.
- ✓ Efficient use of horizontal links.
- ✓ Balanced load and low latency.
- ✓ Ideal for parallel and distributed systems.



The routing algorithm in a hypertree network uses the Least Common Ancestor strategy to provide optimal, scalable, and efficient communication.



Hypertree routing uses a combination of upward, horizontal, and downward moves to deliver messages efficiently along the shortest path.

8. Performance Analysis

Performance analysis of a hypertree network evaluates its efficiency in terms of communication cost, latency, bandwidth usage, and scalability. The hierarchical structure with horizontal links significantly improves performance compared to simple tree or linear topologies.

Key Aspects of Performance

1 Hop Count



- Number of links traversed between source and destination.
- Hypertree reduces hop count using horizontal links.
- For a hypertree of height h , maximum hops = $2h$.

2 Communication Distance



- Total number of links used in a path.
- Fewer horizontal hops reduce the overall distance.
- Average distance in hypertree is lower than binary tree.

3 Latency



- Time taken for a message to travel from source to destination.
- Depends on hop count and link delay.
- $\text{Latency} \propto \text{Hop Count} \times \text{Link Delay}$.

4 Bandwidth Utilization



- Horizontal links provide multiple alternate paths.
- Better load balancing and higher bandwidth utilization.
- Reduces congestion near the root.

5 Scalability



- Number of nodes $N = 2^{h+1} - 1$ for height h .
- Easy to expand by adding leaves at the last level.
- Maintains performance as the network grows.

Comparative Performance (Height $h = 3$)

Topology	Max Hops (S to D)	Avg Hops	Total Links	Remarks
Linear Chain (15 nodes)	14	7.5	14	High delay, poor scalability
Binary Tree (15 nodes)	6	4.0	14	No horizontal links
Hypertree (15 nodes)	6	3.2	20	Horizontal links improve performance

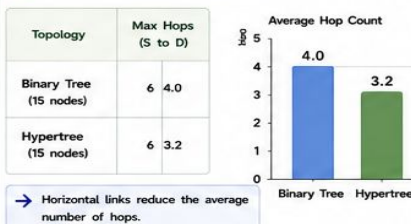
Benefits of Hypertree

- ✓ Lower average hop count.
- ✓ Reduced communication latency.
- ✓ Better bandwidth utilization.
- ✓ Higher reliability with alternate paths.
- ✓ Well suited for parallel and distributed computing environments.
- ✓ Scalable and efficient for large systems.

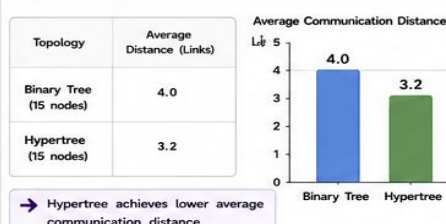
Figure 4.8 Performance Metrics

Performance metrics for a hypertree of height $h = 3$ (15 nodes). Comparison is made with a binary tree of the same number of nodes.

1 Hop Count (Max and Average)



2 Communication Distance (Total Links in Path)



3 Latency

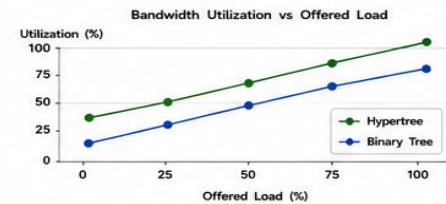
Assume: Link delay = d microseconds

$$\text{Latency} = \text{Hop Count} \times d$$

Topology	Max Latency ($2h \times d$)	Avg Latency (Avg Hops $\times d$)
Binary Tree (15 nodes)	$6d$	$4.0d$
Hypertree (15 nodes)	$6d$	$3.2d$

Lower average hops result in lower average latency.

4 Bandwidth Utilization



5 Scalability (Number of Nodes)

Height h	Nodes in Hypertree ($N = 2^{h+1} - 1$)	Growth Factor
1	3	—
2	7	2.33x
3	15	2.14x
4	31	2.07x
5	63	2.03x
...	...	$\rightarrow 2x$

The number of nodes approximately doubles with every increase in height, while maintaining low communication cost.

6 Summary of Benefits

- ✓ Lower average hop count and communication distance.
- ✓ Reduced latency and higher bandwidth utilization.
- ✓ Multiple alternate paths increase reliability.
- ✓ Scalable structure suitable for large parallel systems.
- ✓ Optimized for high performance and efficient routing.



Overall, the hypertree network provides superior performance by combining the advantages of hierarchical organization and same-level connectivity.



Hypertree networks deliver better performance through reduced hops, lower latency, higher bandwidth utilization, and excellent scalability compared to traditional trees.

9. Advantages

Hypertree networks offer several advantages over simple trees and other interconnection topologies.



1. Lower Average Hop Count

The presence of horizontal links at each level reduces the number of hops between any two nodes, resulting in lower communication delay.



2. Reduced Communication Distance

Horizontal links connect nodes at the same level, which shortens the overall communication distance between source and destination.



3. Higher Bandwidth Utilization

Multiple alternate paths are available due to horizontal links, leading to better load balancing and higher bandwidth utilization.



4. Improved Reliability and Fault Tolerance

Failure of a link or node does not disconnect the network. Alternate paths ensure continuous communication.



5. Good Scalability

Hypertree can be easily expanded by adding new levels (leaves) without affecting the existing structure significantly.



6. Suitable for Parallel and Distributed Systems

Provides efficient communication with low latency, making it ideal for parallel computing and large-scale distributed environments.

Overall, hypertree networks provide efficient, scalable, and reliable communication for large systems.

11. Applications

Hypertree networks are well suited for systems that require high performance, scalability, and reliability.



1. Parallel Processing Systems

Used in multiprocessor systems where processors need fast and efficient communication.



2. Distributed Computing Systems

Supports communication among distributed nodes in clusters and grids.



3. Database Systems

Efficient data transfer between multiple database servers and storage units.



4. High Performance Computing (HPC)

Ideal for supercomputers and scientific computing applications.



5. Communication Networks

Used in backbone networks and routers to interconnect multiple systems.



6. Educational and Research Systems

Useful for academic clusters, simulation environments, and research infrastructures.



Hypertree networks are widely used in areas where performance, scalability, and fault tolerance are critical.

10. Disadvantages

Despite its advantages, hypertree networks also have some limitations.



1. Higher Cost

The addition of horizontal links at every level increases the number of physical links, leading to higher hardware and cabling cost.



2. More Complex Design

The network structure is more complex than a simple tree, requiring careful planning and configuration.



3. Increased Number of Links

The number of links grows significantly compared to a binary tree, which increases installation and maintenance effort.



4. Higher Power Consumption

More links and active components lead to higher power consumption in the system.



5. Difficult Routing in Very Large Networks

Although routing is efficient, managing and routing in very large hypertree networks can become complex.



The increased cost and complexity are the main trade-offs for better performance.

12. Comparison with Binary Tree and Hypercube

Hypertree combines the hierarchical structure of a tree with horizontal links, offering better performance than binary tree and many advantages over hypercube.

Feature	Binary Tree	Hypertree	Hypercube
Structure	Hierarchical (parent-child links only)	Hierarchical + horizontal links at each level	n-dimensional cube connections
Number of Links (Approx.)	$N - 1$	$> N - 1$ (Extra horizontal links)	$(n \times 2^{n-1}) / 2$ (where $N = 2^n$)
Average Hop Count	High ($\approx 2 \log_2 N$)	Lower than Binary Tree	Low ($\approx n / 2$)
Communication Distance	High	Lower	Low
Bandwidth Utilization	Low	High (due to multiple alternate paths)	High
Scalability	Good	Very Good	Good (but complex for large n)
Cost & Complexity	Low	Moderate to High	High (many connections)
Reliability	Low (failure can isolate subtrees)	High (alternate paths available)	High
Best Suited For	Small systems, simple applications	Large parallel systems, distributed systems	Systems requiring very low diameter

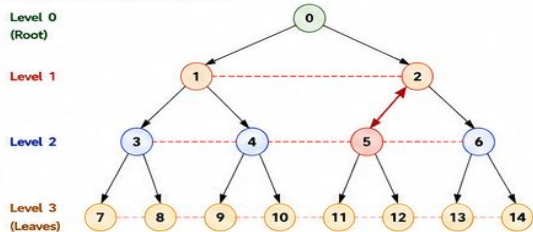


Hypertree provides a good balance between performance, cost, and scalability, making it a preferred choice for large-scale parallel and distributed systems.

13. Worked Example

Consider a hypertree of height $h = 3$ (15 nodes). Find the shortest path, hop count, and communication distance between Node 5 and Node 12.

Step 1: Hypertree Structure



Step 4: Communication Details

Path	Link Type	Hop Count	Description
5 → 2	Vertical (Up)	1	Move from child to parent
2 → 6	Horizontal (Same Level)	1	Move across at LCA level
6 → 12	Vertical (Down)	1	Move from parent to child
Total	-	3	Shortest path between 5 and 12

Step 2: Find LCA

Path from 5 to root: 5 → 2 → 0
Path from 12 to root: 12 → 6 → 2 → 0

Least Common Ancestor (LCA) = Node 2

Step 3: Construct Path

Route: 5 ↑ 2 → 6 ↓ 12

- Move up from 5 to LCA (2): 5 → 2 (1 hop)
- Move across at LCA level: 2 → 6 (1 hop)
- Move down from 6 to 12: 6 → 12 (1 hop)

Total Hops = 3

Step 5: Results

Shortest Path: 5 → 2 → 6 → 12

Hop Count: 3

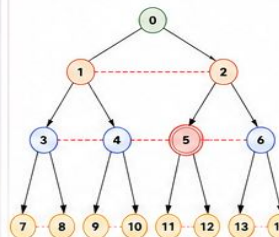
Communication Distance: 3 links

Hypertree routing uses the LCA strategy. The path goes up to the LCA, across at the same level, and down to the destination, resulting in the minimum number of hops.

Figure 4.10 Worked Example

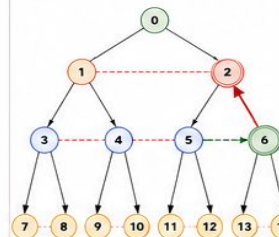
Shortest path from Node 5 to Node 12 in a hypertree of height $h = 3$.

1 Move Up: 5 → 2



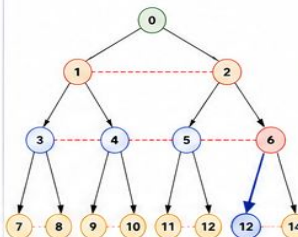
Move from Node 5 up to its parent Node 2.

2 Move Across: 2 → 6



Move horizontally from Node 2 to Node 6 at the same level.

3 Move Down: 6 → 12



Move down from Node 6 to its child Node 12.

Summary of Path



Shortest Path	5 → 2 → 6 → 12
Total Hops	3
Communication Distance	3 links

Legend

- ↑ Move Up (Vertical)
- Move Across (Horizontal)
- ↓ Move Down (Vertical)
- Source / Destination

14. Real-World Case Study

Case Study: Hypertree Interconnection Network in a High-Performance Computing (HPC) System

Problem

An HPC system needs to interconnect thousands of processors with:

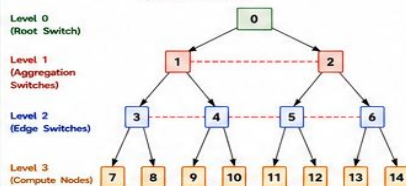
- Low communication latency
- High bandwidth utilization
- Scalability to support system growth
- High reliability and fault tolerance

Solution: Hypertree Network

A hypertree network is used to connect processors in a hierarchical manner with horizontal links at each level.



System Example



- Height (h) = 3, Total Nodes = 15
- Horizontal links connect switches at the same level.
- Compute nodes (level 3) connect to edge switches.
- Data is routed using LCA-based shortest path.

Benefits Observed

- ✓ Reduced average hop count and latency
- ✓ Higher bandwidth utilization due to multiple alternate paths
- ✓ System scales easily by adding new levels
- ✓ High reliability with alternate routing options

15. Summary

The hypertree network is a hierarchical interconnection topology with horizontal links at each level, which improves performance over simple tree structures.

Key Takeaways

- 1 Hypertree is a hierarchical network with horizontal links at each level.
- 2 It reduces the average hop count and communication distance.
- 3 Provides higher bandwidth utilization and multiple alternate paths.
- 4 Routing is based on the Least Common Ancestor (LCA) strategy.
- 5 It is scalable, reliable, and well suited for parallel and distributed systems.
- 6 Horizontal links improve performance but increase cost and complexity.
- 7 Effective for large systems such as HPC, data centers, and clusters.
- 8 Offers a good balance between performance, cost, and scalability.

Quick Facts

Structure	Hierarchical + Horizontal Links
Total Nodes (Height h)	$N = 2^{h+1} - 1$
Average Hop Count	Lower than binary tree
Bandwidth Utilization	High (multiple alternate paths)
Routing Strategy	LCA (Up → Across → Down)
Scalability	Excellent
Best Suited For	Large parallel and distributed systems



Hypertree interconnection in HPC systems enables efficient communication between thousands of compute nodes, resulting in faster computations and better scalability.



Hypertree networks provide efficient, scalable, and reliable communication for modern large-scale computing and communication systems.